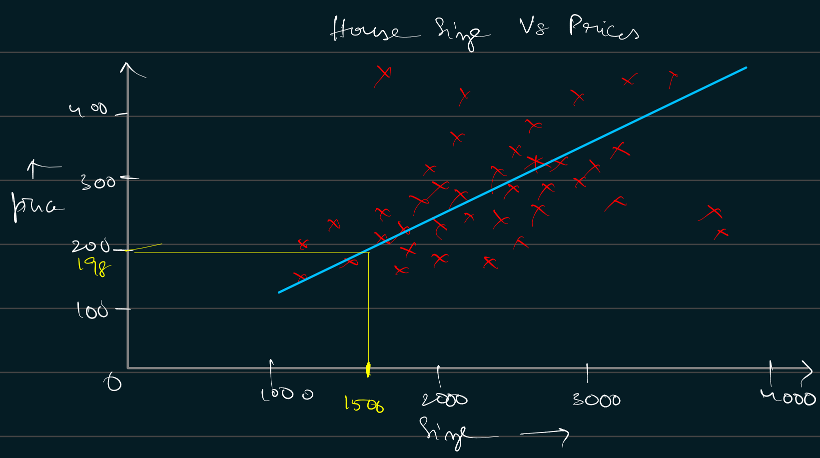
* Linear regression :-



Data table

	Size in feet^2 (x)	Price in \$ 1000 (y)
1)	2104	400
2)	1416	232
3)	1534	315
4)	854	178
⋮	⋮	⋮
47)	3210	870

→ Training data set.

$x \rightarrow$ input variable / feature

$y \rightarrow$ output variable / target variable

$m \rightarrow$ no. of training examples

$(x, y) \rightarrow$ Single training example

$(2104, 400)$

$(x^{(i)}, y^{(i)}) \rightarrow$ i^{th} training example

$x^{(2)} = 1416$

$\boxed{\text{Training Set}}$ features targets

↓

$\boxed{\text{Learning algorithm}}$

↓

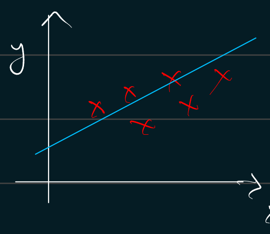
(feature) $x \rightarrow \boxed{f/h} \rightarrow \hat{y}$ (estimate / prediction for y) target

$f \rightarrow$ hypothesis / function / model.

How to represent f ?

lets assum f is a st. line.

$$f_{w,b}(x) = wx + b$$



$f(x) = wx + b$

Linear regression with one variable / feature x. (or)

Univariate linear regression.

→ for multiple input features, for ex.

X_1 = Size

X_2 = no. of bed rooms

$$h(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

$$\Updownarrow$$

$$h(x) = \sum_{j=0}^{n=2} \theta_j x_j$$

where $x_0 = 1$

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \end{bmatrix} \qquad x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix}$$

↓

parameters

m → # training examples
(# rows in table above)

x → input / features

y → output / target variable

(x, y) → training example

$(x^{(i)}, y^{(i)})$ → ith training example

n → no. of features

⇒ choose 'θ' such that $h(x) \simeq y$ for training examples.

$h_\theta(x) = h(x)$

Linear regression also called, ordinary least squares

minimize $\frac{1}{2} \sum_{i=1}^{m} \left(h_\theta(x) - y\right)^2$

Cost function, $J(\theta) = \frac{1}{2} \sum_{i=1}^{m} \left(h_\theta(x) - y\right)^2$

Find parameters, θ that minimize cost function, $J(\theta)$

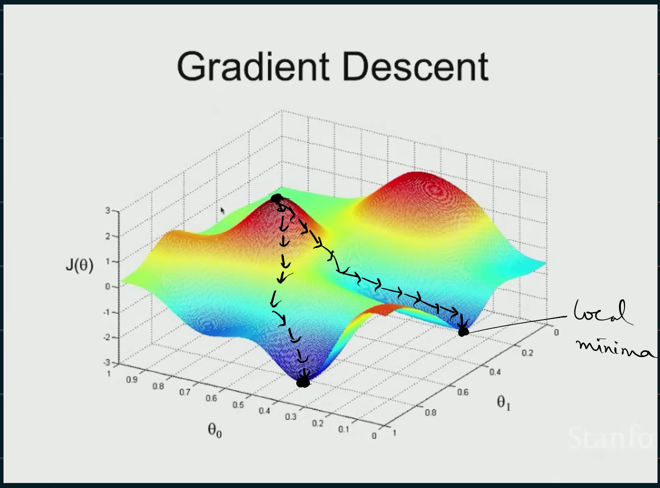
* Gradient descent :-

Algorithm to minimize the cost function.

→ Start with some θ (Say $\theta = \bar{0}$)

→ keep changing θ to reduce $J(\theta)$

→ Disadvantage is that we might go to the local optima instead.



→ for each value of j

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

α → learning rate

$j \to 0, 1, 2, \ldots, n$

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{\partial}{\partial \theta_j} \frac{1}{2} \left(h_\theta(x) - y \right)^2$$ Consider only 1 training example

$$= \cancel{2 \cdot \frac{1}{2}} \left(h_\theta(x) - y \right) \cdot \frac{\partial}{\partial \theta_j} \left(h_\theta(x) - y \right)$$

$$= \left(h_\theta(x) - y \right) \frac{\partial}{\partial \theta_j} \left(\theta_0 x_0 + \underbrace{\theta_1 x_1} + \dots + \theta_n x_n - y \right)$$

if $j=1 \Rightarrow \frac{\partial}{\partial \theta_1} (\theta_1 x_1) = x$

$$\frac{\partial}{\partial \theta_j} J(\theta) = \left(h_\theta(x) - y \right) \cdot x_j$$

$$\boxed{\frac{\partial}{\partial \theta_j} J(\theta) = \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right) \cdot x_j^{(i)}}$$ → for multiple training examples

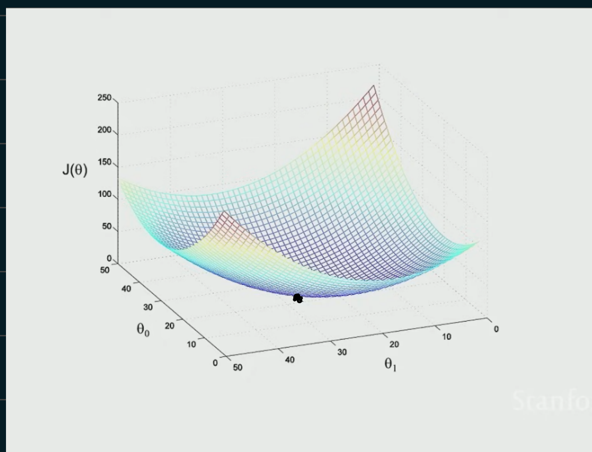
$$\Rightarrow \boxed{\theta_j := \theta_j - \sum_{i=1}^{m} \alpha \left(h_\theta(x^{(i)}) - y^{(i)} \right) \cdot x_j^{(i)}}$$ $m \rightarrow$ no. of training examples

↓

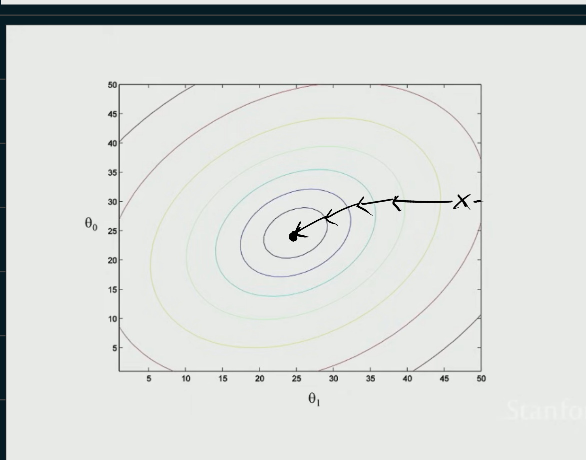
Repeat until convergence

(for $j = 0, 1, 2, \dots, n$ no. of features)

→ Now, $J(\theta)$ becomes a quadratic funct. with only 1 local optima which is also the global optima.



Contour of the plot ←



α (learning rate) → Step size

→ when taken too big, it overshoots & run past minimum.

→ when taken too small, need more iterations & algo is slow.

→ It is also called as 'Batch gradient descent'. Refers to that the learning algo has to go through all the learning examples to process that (batch of data)

→ Disadvantage of batch gradient descent in order to make an update we have to scan through the complete (large) data set. $\left[\sum_{i=1}^{m} \left(h(x^{(i)}) - y^{(i)} \right) \right]$. So, it becomes slow.

✱ Stochastic gradient descent :-

Repeat {

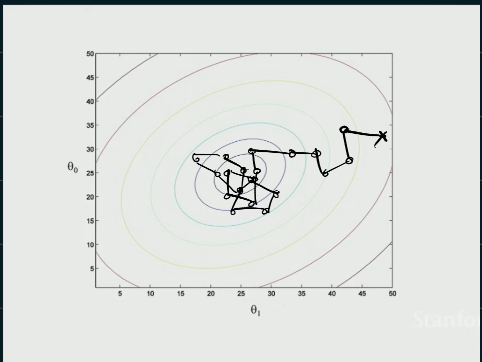
for $i=1$ to m {

$$\theta_j := \theta_j - \alpha \left(h_\theta(x^{(i)}) - y^{(i)} \right) \cdot x_j^{(i)}$$

}

}

→ Instead of scanning through all the dataset to update, stocastic gradient descent loops through $i=1$ to m finding the derivate of one single example & updates it.



✱ Normal equation :-

→ Used only for linear regression model

$$\nabla_\theta J(\theta) = \begin{bmatrix} \frac{\partial J}{\partial \theta_0} \\ \frac{\partial J}{\partial \theta_1} \\ \frac{\partial J}{\partial \theta_2} \end{bmatrix} \rightarrow \text{for 3 components}$$

$\theta \in \mathbb{R}^{n+1}$

Suppose, $A = \mathbb{R}^{2\times 2}$

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$$

then a fund., f

$$\underline{f(A) = A_{11} + A_{12}^2} \qquad f : \mathbb{R}^{2\times 2} \to \mathbb{R}$$

$$f\left(\begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}\right) = 5 + 6^2$$

$$\nabla_A f(A) = \begin{bmatrix} \frac{\partial f}{\partial A_{11}} & \frac{\partial f}{\partial A_{12}} \\ \frac{\partial f}{\partial A_{21}} & \frac{\partial f}{A_{22}} \end{bmatrix} \qquad \text{if } A = [\;]_{2\times 2}$$

$$\nabla_A f(A) = \begin{bmatrix} 1 & 2A_{12} \\ 0 & 0 \end{bmatrix}$$

$\nabla_\theta J(\theta) = 0$ (to find minima of θ)

If A is a sq. matrix ($A \in \mathbb{R}^{n\times n}$)

trace(A) = sum of diag. entries

$= \sum_{i=0}^{n} A_{ii}$.

$\rightarrow$ $\mathrm{tr}\,A = \mathrm{tr}\,A^T$

$\rightarrow$ if $f(A) = \mathrm{tr}\,AB$

then $\nabla_A f(A) = B^T$

$\rightarrow$ $\mathrm{tr}\,AB = \mathrm{tr}.\,BA$.

$\rightarrow$ $\mathrm{tr}.\,ABC = \mathrm{tr}.\,CAB$

$\rightarrow$ $\nabla_A \mathrm{tr}.\,AA^TC = CA + C^TA$

$\Downarrow$ analogous to

$\frac{d}{da} a^2c = 2ac$

$$J(\theta) = \frac{1}{2} \sum_{i=1}^{m} \left(h(x^{(i)}) - y^{(i)} \right)^2$$

Let

$$X = \begin{bmatrix} (x^{(1)})^T \\ (x^{(2)})^T \\ \vdots \\ (x^{(m)})^T \end{bmatrix}$$

$\downarrow$

Design matrix

$$X\theta = X \cdot \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \end{bmatrix}$$

$$X\theta = \begin{bmatrix} (x^{(1)})^T \theta \\ (x^{(2)})^T \theta \\ \vdots \\ (x^{(m)})^T \theta \end{bmatrix} = \begin{bmatrix} h_\theta(x^{(1)}) \\ h_\theta(x^{(2)}) \\ \vdots \\ h_\theta(x^{(m)}) \end{bmatrix}$$

$$\bar{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix}$$

$$\boxed{J(\theta) = \frac{1}{2} (X\theta - y)^T (X\theta - y)}$$

$$(X\theta - y) = \begin{bmatrix} h(x^{(1)}) - y^{(1)} \\ \vdots \\ h(x^{(m)}) - y^{(m)} \end{bmatrix}$$

$$z^T z = \sum z^2$$

So,

$$(X\theta - y)^T (X\theta - y) = \text{Sum of sqrs of elements of } (X\theta - y)$$

So,

$$J(\theta) = \frac{1}{2} \underbrace{\sum_{i=1}^{m} \left(h(x^{(i)}) - y^{(i)} \right)^2}_{(X\theta - y)^T (X\theta - y)}$$

$$\boxed{J(\theta) = \frac{1}{2} (X\theta - y)^T (X\theta - y)}$$

$$\nabla_\theta J(\theta) = \nabla_\theta \frac{1}{2} (X\theta - y)^T (X\theta - y) = 0$$

$$= 0$$

$$= \frac{1}{2} \nabla_\theta (\theta^T X^T - y^T)(X\theta - y) = 0$$

$$= \frac{1}{2} \nabla_\theta \left[\theta^T X^T X\theta - \theta^T X^T y - y^T X\theta + y^T y \right] = 0$$

$$\left[(ax - b)(ax - b) = a^2 x^2 - axb - bax + b^2 \right]$$

$$= \frac{1}{2} \left[X^T X\theta + X^T X\theta - X^T y - X^T y \right] = 0$$

$$= X^T X\theta - X^T y = 0$$

$$= X^T X\theta = X^T y \longrightarrow \text{normal equation}$$

$$\boxed{\theta = (X^T X)^{-1} X^T y}$$

what if X is non invertable?

We have some redundant features (features are linearly dependent).

Psendo inverse can be used but it's better to remove the features that repeated.